

Blockchain Wallet Setup Guide

☞ Wallet Initialization Location

****File**:** `functions/src/services/blockchain.ts`
****Line**:** 32

Code Flow

```
```typescript
// 1. Global configuration from environment variables (Lines 3-6)
const BLOCKCHAIN_RPC = process.env.BLOCKCHAIN_RPC || "";
const BLOCKCHAIN_PRIVATE_KEY = process.env.BLOCKCHAIN_PRIVATE_KEY || "";
const CONTRACT_ADDRESS = process.env.CONTRACT_ADDRESS || "";
const BLOCKCHAIN_ENABLED = process.env.BLOCKCHAIN_ENABLED === 'true';

// 2. Global wallet instances (Lines 15-17)
let provider: ethers.providers.JsonRpcProvider | null = null;
let signer: ethers.Wallet | null = null; // ← Wallet created here
let contract: ethers.Contract | null = null;

// 3. Initialization function (Lines 19-43)
function initializeBlockchain(): void {
 // ...
 provider = new ethers.providers.JsonRpcProvider(BLOCKCHAIN_RPC);
 signer = new ethers.Wallet(BLOCKCHAIN_PRIVATE_KEY, provider); // ← WALLET
 CREATED
 contract = new ethers.Contract(CONTRACT_ADDRESS, CONTRACT_ABI, signer);
 // ...
}
```
```

🗝 Environment Variables Setup

Step 1: Update `.env` File

Edit `functions/.env` and add blockchain configuration:

```
```env
PostgreSQL Configuration (existing)
PG_HOST=34.143.210.224
PG_PORT=5432
PG_USER=postgres
PG_PASSWORD=108bB2212!2025
PG_DB=low_emission_mobility

Blockchain Configuration (NEW)
BLOCKCHAIN_RPC=https://mainnet.infura.io/v3/YOUR_INFURA_PROJECT_ID
BLOCKCHAIN_PRIVATE_KEY=your_wallet_private_key_here
CONTRACT_ADDRESS=0x...
BLOCKCHAIN_ENABLED=true
```
```

Step 2: Set Firebase Functions Configuration

Run these commands to set blockchain environment variables:

```
```bash
cd functions

Set blockchain RPC
firebase functions:config:set
blockchain.rpc="https://mainnet.infura.io/v3/YOUR_KEY"

Set private key
firebase functions:config:set blockchain.privatekey="YOUR_PRIVATE_KEY"

Set contract address
firebase functions:config:set blockchain.contractaddress="0x..."

Enable blockchain
firebase functions:config:set blockchain.enabled="true"

Verify configuration
firebase functions:config:get
```

---
```

☞ How Wallet Initialization Works

Flow Diagram

```
```
1. Deploy Firebase Functions
 ↓
2. initializeBlockchain() is called
 ↓
3. Create JsonRpcProvider (connects to blockchain RPC)
 ↓
4. Create Wallet from BLOCKCHAIN_PRIVATE_KEY
 ↓
5. Create Smart Contract instance with signer
 ↓
6. Log wallet address and contract address
 ↓
7. Ready for blockchain transactions
```
```

Detailed Code Explanation

```
```typescript
// Step 1: Load environment variables
const BLOCKCHAIN_PRIVATE_KEY = process.env.BLOCKCHAIN_PRIVATE_KEY || "";

// Step 2: Create provider (RPC connection)
```

```

provider = new ethers.providers.JsonRpcProvider(BLOCKCHAIN_RPC);

// Step 3: Create wallet from private key + provider
// This creates a new ethers.Wallet instance
signer = new ethers.Wallet(BLOCKCHAIN_PRIVATE_KEY, provider);

// The wallet automatically derives:
// - Public key from private key
// - Ethereum address from public key
// - Signer instance for transactions

// Access wallet address
console.log(signer.address); // e.g., 0x1234...

// Step 4: Create contract with signer for signing transactions
contract = new ethers.Contract(CONTRACT_ADDRESS, CONTRACT_ABI, signer);
` ``

```

---

## ## 📖 Wallet Properties

Once initialized, the wallet (`signer`) has these properties:

```

` ``typescript
// Wallet Address
signer.address // e.g., "0x1234...5678"

// Get Balance
await signer.getBalance() // Returns balance in wei

// Sign Messages
await signer.signMessage(message)

// Sign Transactions
await signer.sendTransaction(tx)
` ``

```

---

## ## 🔗 How to Get Wallet Information

### ### Option 1: View Initialized Wallet Address

Edit `functions/src/index.ts` and add:

```

` ``typescript
import { blockchainHealthCheck } from "../services/blockchain";

export const getBlockchainStatus = onRequest(async (req: Request, res: Response)
=> {
 try {
 const status = await blockchainHealthCheck();
 res.json(status);
 }

```

```

 } catch (error) {
 res.status(500).json({ error: String(error) });
 }
 });
}

```

Deploy and call:

```

```bash
curl https://your-project.cloudfunctions.net/getBlockchainStatus
```

```

Response:

```

```json
{
  "status": "healthy",
  "details": {
    "network": "mainnet",
    "chainId": 1,
    "blockNumber": 18500000,
    "contractAddress": "0x...",
    "signerBalance": "0.5"
  }
}
```

```

### Option 2: Check Console Logs

After deployment, check Firebase Functions logs:

```

```bash
firebase functions:log
```

```

Look for:

```

```
Blockchain initialized: {
  network: "https://mainnet.infura.io/v3/...",
  contractAddress: "0x...",
  signerAddress: "0x1234...5678" ← Your wallet address
}
```

```

---

## 🔗 Creating Blockchain Private Key

### From MetaMask

1. Open MetaMask → Settings → Security & Privacy
2. Click "Export Private Key"
3. Enter password, copy private key
4. Add to ``.env``:
 

```

```env
BLOCKCHAIN_PRIVATE_KEY=your_copied_private_key
```

```

```

From ethers.js

```
```typescript
import { ethers } from "ethers";

// Generate new wallet
const newWallet = ethers.Wallet.createRandom();
console.log("Private Key:", newWallet.privateKey);
console.log("Address:", newWallet.address);
```
```

📋 Required for Production Deployment

Checklist

- [] Have RPC endpoint URL (Infura, Alchemy, Quicknode, etc.)
- [] Have wallet private key
- [] Have deployed smart contract address
- [] Set `BLOCKCHAIN\_ENABLED=true`
- [] Test wallet has sufficient ETH for gas fees
- [] Smart contract has required functions:
 - `redeemVoucher(address user, string voucherId)`
 - `getRedemption(address user, string voucherId)`

Deploy with Blockchain

```
```bash
cd functions

1. Set all environment variables
firebase functions:config:set \
 blockchain.rpc="https://mainnet.infura.io/v3/YOUR_KEY" \
 blockchain.privatekey="YOUR_PRIVATE_KEY" \
 blockchain.contractaddress="0x..." \
 blockchain.enabled="true"

2. Deploy functions
firebase deploy --only functions

3. Verify
firebase functions:log
```
```

🐛 Troubleshooting Wallet Issues

Issue: "Blockchain initialization failed"

****Solution**:** Check environment variables

```
```bash
firebase functions:config:get | grep blockchain
```
```

Make sure all three are set:

```
- `blockchain.rpc`
- `blockchain.privatekey`
- `blockchain.contractaddress`
```

Issue: "Invalid Ethereum address"

****Solution**:** Verify private key format

```
```bash
Should NOT start with 0x
☒ BLOCKCHAIN_PRIVATE_KEY=abc123def456...
☒ BLOCKCHAIN_PRIVATE_KEY=0xabc123def456...
```
```

Issue: "Insufficient funds for gas fees"

****Solution**:** Add ETH to wallet

```
```bash
Get wallet address
firebase functions:log | grep signerAddress

Send ETH to that address from another wallet
Then retry transaction
```
```

Issue: "Transaction nonce expired"

****Solution**:** Retry the transaction

```
```bash
The wallet will use correct nonce on next attempt
No manual intervention needed
```
```

📞 Wallet Functions Available

1. recordRedemptionOnChain()

```
```typescript
const txHash = await recordRedemptionOnChain(
 userAddress, // User's wallet address
 voucherId // Voucher ID to redeem
);
```
```

2. checkRedemptionOnChain()

```
```typescript
const isRedeemed = await checkRedemptionOnChain(
```

```

 userAddress, // User's wallet address
 voucherId // Voucher ID to check
);
 ...

```

### ### 3. getAccountBalance()

```

```typescript
const balance = await getAccountBalance();
// Returns balance in ETH (formatted)
```

```

### ### 4. blockchainHealthCheck()

```

```typescript
const status = await blockchainHealthCheck();
// Returns: { status, details }
```

```

---

## ## 📁 Wallet Lifecycle

...

Application Start

↓

initializeBlockchain() called (first function using blockchain)

↓

Provider created

↓

Wallet created from BLOCKCHAIN\_PRIVATE\_KEY

↓

Contract instance created with signer

↓

Wallet ready for signing transactions

↓

Transaction executed with wallet signature

↓

Blockchain records transaction

↓

Transaction receipt returned to application

...

---

## ## 📖 References

- **ethers.js Wallet**: <https://docs.ethers.org/v5/api/signer/#Wallet>
- **Firebase Functions Config**:  
<https://firebase.google.com/docs/functions/config-env>
- **Infura RPC**: <https://infura.io/>
- **Smart Contract ABI**: <https://docs.ethers.org/v5/api/contract/contract-abi/>

---